# Indor Robot Pose Estimation

Eduardo Henrique Basilio de Carvalho

***Abstract*—This report presents the development and evaluation of various algorithms for estimating the pose of the Indor robot using laser and odometry data. Different filtering techniques, including the Unscented Kalman Filter, Extended Kalman Filter, and Particle Filter, were implemented and compared. The performance of each method was assessed based on accuracy and computational efficiency. Hyperparameter optimization was conducted to enhance the models' performance. The results highlight the importance of the model selection for effective pose filtering in the studied problem.**

## I. Introduction

In the provided problem, the odometry data exhibits a noticeable drift, as discussed in Section III-C. This drift leads to a significant discrepancy between the odometry estimates and the reference data, which can adversely affect the overall pose estimation accuracy. While laser data has the potential to mitigate this drift by providing more accurate position information, it is important to note that laser readings are often subject to noise and other environmental factors. Consequently, relying solely on laser data for robot localization presents its own challenges. The integration of both odometry and laser data is essential for achieving a robust and reliable pose estimation, as it allows for the correction of odometry drift while leveraging the strengths of each data source.

Since the trajectory spans across several meters and the laser model exhibits dependency on the map structure, linear filtering approaches prove insufficient for achieving adequate pose estimates. The spatial variation of the environment introduces nonlinearities in the measurement model that cannot be adequately captured by linear filters such as the standard Kalman Filter. Consequently, nonlinear filtering techniques are necessary to properly account for these dynamics and provide reliable pose estimation [1]. These advanced filtering methods are discussed in detail in Section VII.

## II. Map

Figure 1 illustrates the environment in which the robotic experiments were conducted. The map is represented as a binary occupancy grid, where white pixels denote free space accessible to the robot, while black pixels represent obstacles and walls that must be avoided during navigation. This map representation is essential for the laser model, as it defines the expected measurement relationships between the robot's pose and the corresponding laser range observations.

### A. Transformation to Data's Frame

The source material provides the map in conventional coordinates with pixel-scale representation. However, since the experimental data is provided in a shifted, rotated, and scaled reference frame, several coordinate transformations were necessary to ensure consistency between the map and the data. The following transformations were applied in sequence:

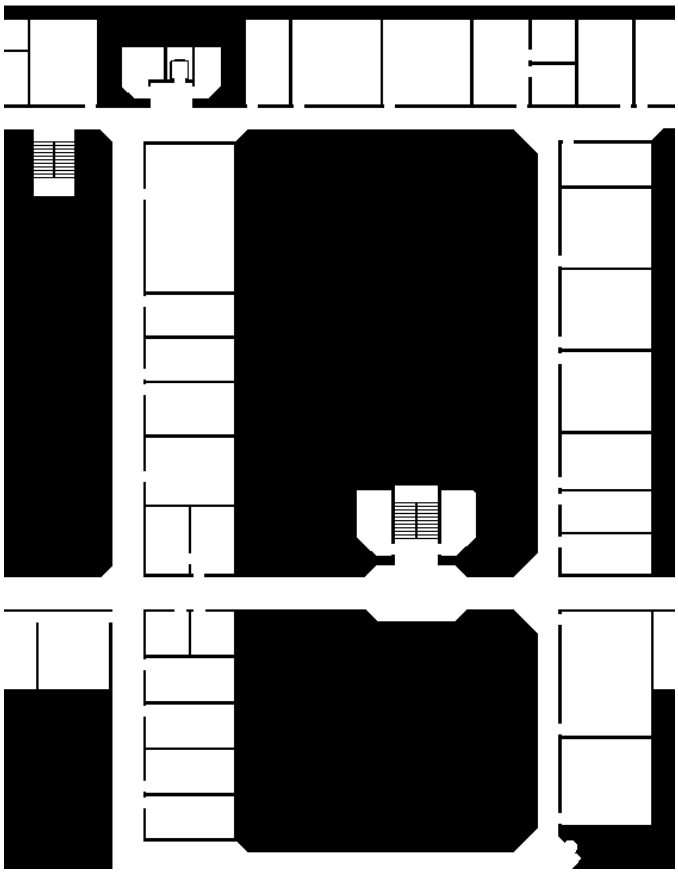

Fig. 1. Map of the environment where the Indor robot operates.

1) **Origin Centering:** The coordinate system was translated such that the origin $(0, 0)$ is positioned at the center of the map, rather than at a corner. This transformation is defined as:

$$x' = x - \frac{w}{2}, \quad y' = y - \frac{h}{2} \tag{1}$$

where $w$ and $h$ are the width and height of the map in pixels, respectively.

2) **Y-axis Inversion:** The $y$-axis was inverted to align with the data's coordinate convention, transforming the pixel coordinates from image space to a standard Cartesian frame:

$$y'' = -y' \tag{2}$$

3) **Scale Conversion:** Pixel coordinates were converted to metric units using the provided scale factor $s$ (in meters per pixel):

$$x_m = s \cdot x', \quad y_m = s \cdot y'' \tag{3}$$

4) **Spatial Cropping:** The map was cropped to focus on the region of interest encompassing the experimental trajectory, reducing computational overhead while maintaining the necessary environmental context for laser-based localization.

These transformations ensure that the map representation is consistent with the odometry and laser data frames, enabling accurate pose estimation throughout the experiment.

## III. Data

The experimental data was initially provided in a single space-separated file, which contained various measurements and observations relevant to the pose estimation task. To enhance organization and facilitate subsequent processing, this data was systematically divided into three distinct comma-separated files. Each file corresponds to a specific type of data: reference data, laser measurements, and odometry readings.

This separation allows for more efficient data handling and processing, as each file can be independently accessed and manipulated according to its specific requirements. The reference data file contains the ground truth positions against which the estimated poses can be compared. The laser measurements file includes the range and bearing information captured by the robot's laser sensors, while the odometry readings file records the robot's estimated position based on its movement and wheel encoders.

If we denote the original data as $D$ and the separated files as $D_{ref}$, $D_{laser}$, and $D_{odom}$, we can express the separation process as follows:

$$D = \{D_{ref}, D_{laser}, D_{odom}\}$$

This structured approach not only improves clarity but also streamlines the data preprocessing steps that follow, ensuring that each dataset can be effectively utilized in the filtering and estimation algorithms discussed in subsequent sections.

### A. Reference

The reference data is provided in the global frame with absolute values, which is crucial for accurate pose estimation. This dataset consists of three columns: the first column contains the $x$ coordinates, representing the robot's position along the horizontal axis; the second column contains the $y$ positions, indicating the robot's position along the vertical axis; and the third column contains the yaw angles, denoted as $\theta$, which represent the orientation of the robot in radians.

Mathematically, we can express the reference data as a set of tuples $R$ defined as follows:

$$R = \{(x_i, y_i, \theta_i) \mid i = 1, 2, \ldots, N\}$$

where $N$ is the total number of observations. Each tuple $(x_i, y_i, \theta_i)$ corresponds to a specific time step $i$ and provides essential information for evaluating the performance of the pose estimation algorithms. The yaw angle $\theta_i$ is particularly important as it allows for the transformation of the robot's pose from the global frame to the local frame, facilitating the integration of laser and odometry data in subsequent filtering processes.

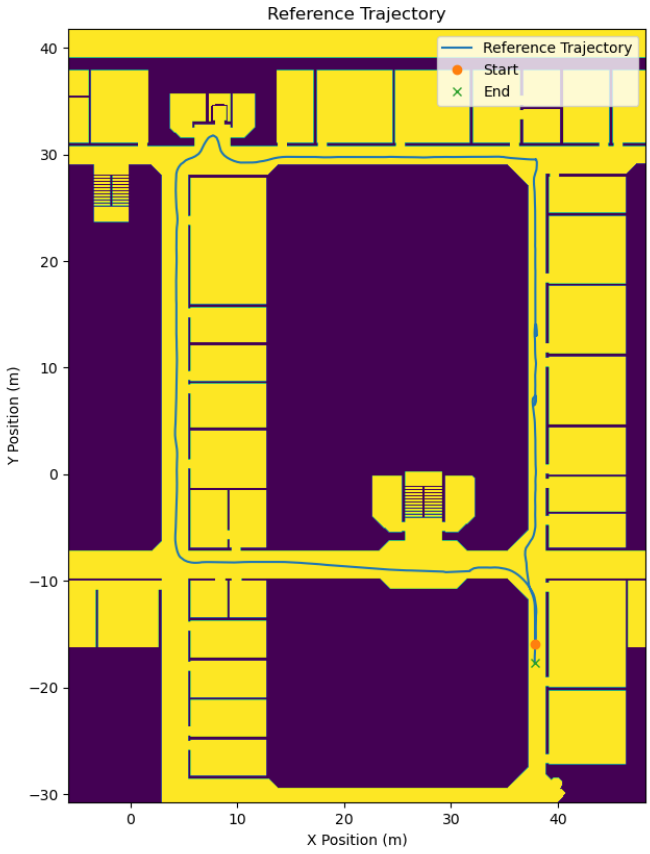


Fig. 2. Reference trajectory of the Indor robot in the global frame.

Figure 2 presents a visual representation of the reference data plotted over the map of the environment. This overlay allows for a direct comparison between the estimated poses derived from the filtering algorithms and the ground truth positions provided in the reference dataset.

Mathematically, we can denote the reference positions as $R = \{(x_i, y_i, \theta_i) \mid i = 1, 2, \ldots, N\}$, where each tuple corresponds to the robot's position and orientation at discrete time steps. The plotted reference trajectory serves as a benchmark for evaluating the accuracy of the pose estimation methods employed in this study.

The visualization not only highlights the trajectory of the Indor robot but also emphasizes the spatial relationship between the robot's estimated poses and the obstacles present in the environment. This context is crucial for understanding the performance of the filtering techniques, as it illustrates how

well the algorithms navigate through the mapped space while adhering to the constraints imposed by the environment.

In the subsequent sections, we will analyze the discrepancies between the reference data and the estimated poses, providing insights into the effectiveness of the various filtering approaches utilized in this research.

### B. Laser

The laser data comprises 360 columns, each corresponding to a single laser sensor reading. These sensors are angularly displaced by 0.5 degrees from one another, collectively covering the 180-degree frontal hemisphere of the robot. Mathematically, the angular coverage can be expressed as:

$$\Delta\phi = 0.5\,\text{deg}, \quad \Phi_{total} = 360 \times 0.5\,\text{deg} = 180\,\text{deg} \tag{4}$$

where $\Delta\phi$ denotes the angular separation between adjacent sensors and $\Phi_{total}$ represents the total angular coverage.

Each sensor returns a range measurement in meters, representing the distance to the nearest obstacle detected in its corresponding direction. These measurements are expected to fall within the range $[0, 8]$ meters, defining the operational range of the laser scanner. The laser data can be represented as a vector of measurements $L$ at each time step:

$$L = \{r_j \mid r_j \in [0, 8]\ \text{m}, \quad j = 1, 2, \ldots, 360\}$$

where $r_j$ denotes the range reading from the $j$-th sensor. This rich angular resolution provides detailed geometric information about the robot's surroundings, which is essential for accurate position correction in the pose estimation algorithms discussed in Section V.

Figure 3 illustrates a sample laser scan taken at a specific time step. In this figure, each red dot represents a laser measurement projected into the environment based on the robot's current pose. The distribution of these measurements provides insight into the spatial configuration of obstacles around the robot, which is crucial for effective localization and navigation.

In the context of the robot's pose and laser range data, we can express the transformation of laser readings into global Cartesian coordinates mathematically.

Let:

- $(x, y, \theta)$ represent the robot's position and orientation, where $x$ and $y$ are the coordinates in the global frame, and $\theta$ is the heading angle.
- $r_i$ denote the laser range measurement at angle $\alpha_i$ relative to the robot's heading.

The global coordinates of the laser points can be computed as follows:

1) For each laser reading, the global angle is given by:

$$\text{global\_angle}_i = \theta + \alpha_i \tag{5}$$

2) The Cartesian coordinates of the laser point in the global frame are then calculated using:


Fig. 3. Laser scan at a specific time step. Each red dot corresponds to a laser measurement.

$$x_{\text{laser},i} = x + r_i \cdot \cos(\text{global\_angle}_i)$$
$$y_{\text{laser},i} = y + r_i \cdot \sin(\text{global\_angle}_i)$$

This transformation ensures that the laser readings, which are initially in polar coordinates relative to the robot's local frame, are accurately represented in the global Cartesian coordinate system. The filtering condition $0.1 < r_i < 8.0$ ensures that only valid laser readings are considered for this transformation.

### C. Odometry

The odometry data serves as a crucial component in understanding the robot's movement throughout the experiment. It provides the cumulative displacement of the robot from its initial pose, allowing for the assessment of its trajectory over time. Each row in the odometry dataset corresponds to the global displacement from the starting position to a specific moment in the experiment.

Mathematically, we can represent the odometry data as a set of tuples $O$ defined as follows:

$$O = \{(x_i, y_i, \theta_i) \mid i = 1, 2, \ldots, N\}$$

where $N$ is the total number of observations. In this representation, the first column contains the $x$ coordinates, indicating the robot's position along the horizontal axis; the second column contains the $y$ coordinates, representing the robot's position along the vertical axis; and the third column contains the yaw angles, denoted as $\theta$, which indicate the orientation of the robot in radians.

The cumulative displacement can be expressed as:

$$\Delta x = x_i - x_0, \quad \Delta y = y_i - y_0, \quad \Delta\theta = \theta_i - \theta_0$$

where $(x_0, y_0, \theta_0)$ represents the initial pose of the robot. This formulation allows for the calculation of the robot's position and orientation at any given time step $i$ relative to its starting point.

It is important to note that the odometry data serves as the input to the odometry models discussed in Section VI. Rather than providing a reference for evaluation, the odometry model functions as the *process model* [1] in the estimation techniques employed in this work. Specifically, the odometry model generates predictions of the robot's pose evolution that are subsequently corrected by the integration of laser measurements through the filtering algorithms. The predictions from the odometry model are refined by the filtering techniques to account for the accumulated drift and produce more accurate pose estimates. This correction mechanism is central to the effectiveness of the Unscented Kalman Filter, Extended Kalman Filter, and Particle Filter approaches discussed in Section VII.

## IV. Data Preprocessing

Before the main problem of pose estimation is addressed, the experimental data must undergo preprocessing to facilitate subsequent identification and filtering operations. The preprocessing steps are designed to extract a representative subset of the data that captures the essential dynamics of the robotic experiment while reducing computational burden. Formally, if we denote the complete dataset as $D = \{D_{ref}, D_{laser}, D_{odom}\}$ with $N$ total observations, the preprocessing aims to produce a reduced dataset $D' \subset D$ with $N' < N$ observations such that:

$$\mathcal{C}(D') \ll \mathcal{C}(D) \tag{6}$$

where $\mathcal{C}(\cdot)$ represents the computational cost of subsequent filtering and estimation operations, while maintaining sufficient fidelity to the original experimental characteristics.

The preprocessing strategy employed in this work comprises two main steps: data decimation, which reduces the sampling frequency to achieve computational efficiency, and trajectory trimming, which removes the initial and final segments of the trajectory that may contain noise or irrelevant movements. These operations are described in detail in the following subsections.

### A. Decimation

Data decimation is performed to reduce the computational burden of subsequent filtering operations while preserving the essential dynamics of the robot's trajectory [1]. The decimation process selects a subset of the original observations at regular intervals, effectively reducing the sampling frequency. If the original dataset contains $N$ observations sampled at frequency $f_s$, decimation with factor $d$ yields a reduced dataset with $N' = \lfloor N/d \rfloor$ observations sampled at frequency $f'_s = f_s/d$.

The decimation factor was determined through a systematic trial-and-error manual process. Various decimation ratios were tested by applying each factor to the complete dataset and subsequently visualizing the resulting reference trajectory overlaid on the environment map. This visual inspection approach allowed for qualitative assessment of how well the decimated trajectory preserved the essential characteristics of the original path. The selection criterion was to identify the highest decimation factor that yielded a trajectory sufficiently similar to the original, thereby minimizing data while maintaining fidelity to the experimental dynamics.

Following this evaluation procedure, a decimation factor of $d = 100$ was selected as the optimal compromise between computational efficiency and trajectory preservation. This factor reduces the dataset by two orders of magnitude, substantially decreasing the computational requirements for the filtering algorithms discussed in Section VII, while ensuring that the decimated trajectory adequately represents the robot's movement through the environment.

It is important to note that all figures and results presented throughout this document already reflect the application of this decimation factor. Consequently, the visualizations and quantitative analyses shown hereafter correspond to the reduced dataset with sampling frequency $f'_s = f_s/100$.

### B. Trimming

Trajectory trimming is performed to eliminate segments of the robot's path that may introduce noise or irrelevant movements, particularly at the beginning and end of the experiment. These segments often contain transient behaviors, such as manual positioning or stopping maneuvers, which do not contribute meaningfully to the analysis of the robot's steady-state navigation performance. By removing these portions of the trajectory, the focus is placed on the segments where the robot is actively navigating the environment, thereby enhancing the quality of the data used for filtering and estimation. The first 10 and last 50 observations were removed from the dataset, resulting in a trimmed trajectory that better represents the robot's operational behavior during the experiment.

## V. Laser Model

### A. Linear Dynamic Model

The laser-based position estimation task is formulated as a discrete-time linear dynamic system where the robot's state evolves according to laser measurements. The state vector comprises three components: horizontal position, vertical position, and orientation angle. At each time step, the state

transition is governed by a linear relationship between the current state, extracted laser features, and a bias term.

Formally, the model is defined as:

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{f}(\mathbf{s}_k) + \mathbf{b} \tag{7}$$

where:

- $\mathbf{x}_k = [x_k, y_k, \theta_k]^T$ is the state vector at time step $k$
- $\mathbf{A} \in \mathbb{R}^{3\times3}$ is the state transition matrix
- $\mathbf{f}(\mathbf{s}_k) \in \mathbb{R}^6$ is a feature vector extracted from the laser scan
- $\mathbf{B} \in \mathbb{R}^{3\times6}$ is the laser feature weight matrix
- $\mathbf{b} \in \mathbb{R}^3$ is a bias vector

*1) Laser Feature Extraction:* Rather than using all 360 laser measurements directly, six summary statistics are computed to reduce dimensionality while capturing essential geometric information about the robot's surroundings. These features are:

$$\mathbf{f}(\mathbf{s}) = [\mu, r_{\min}, \sigma, \mu_{\text{left}}, \mu_{\text{front}}, \mu_{\text{right}}]^T \tag{8}$$

where:

- $\mu = \frac{1}{360}\sum_{j=1}^{360} r_j$ is the mean of all laser ranges
- $r_{\min} = \min(r_1, r_2, \ldots, r_{360})$ is the minimum range (closest obstacle)
- $\sigma = \sqrt{\frac{1}{360}\sum_{j=1}^{360}(r_j - \mu)^2}$ is the standard deviation
- $\mu_{\text{left}}, \mu_{\text{front}}, \mu_{\text{right}}$ are mean ranges computed over the left (sensors 1–120), front (sensors 121–240), and right (sensors 241–360) sectors respectively

These features capture both global context (mean, minimum, standard deviation) and local directional information about the environment relative to the robot's heading.

*2) Model Identification:* The model parameters $\{\mathbf{A}, \mathbf{B}, \mathbf{b}\}$ are identified from experimental data using linear regression [1]. Given reference trajectory data $\{\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_N\}$ and corresponding laser scans $\{\mathbf{s}_0, \mathbf{s}_1, \ldots, \mathbf{s}_N\}$, the regression problem is formulated as:

$$\min_{\mathbf{A},\mathbf{B},\mathbf{b}} \sum_{k=0}^{N-\Delta} \|\mathbf{x}_{k+\Delta} - (\mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{f}(\mathbf{s}_k) + \mathbf{b})\|_2^2 \tag{9}$$

where $\Delta$ is a time step parameter (typically $\Delta = 1$). This optimization problem is solved using ordinary least squares regression, yielding closed-form solutions for the model parameters.

### *B. Nonlinear AutoRegressive eXogenous (NARX) Model*

To capture nonlinear relationships between laser observations and state evolution, a NARX model is employed [2]. This approach incorporates past state information through an autoregressive structure, allowing the model to learn temporal dependencies in the robot's dynamics.

The NARX model is defined as:

$$\mathbf{x}_{k+1} = g(\mathbf{x}_k, \mathbf{x}_{k-1}, \ldots, \mathbf{x}_{k-n_{\text{lag}}+1}, \mathbf{f}(\mathbf{s}_k)) \tag{10}$$

where $g(\cdot)$ is a nonlinear function parameterized by a machine learning model, and $n_{\text{lag}}$ is the number of past time steps (lags) used as autoregressive inputs.

The input feature vector concatenates past state history and current laser features:

$$\mathbf{u}_k = [\mathbf{x}_k, \mathbf{x}_{k-1}, \ldots, \mathbf{x}_{k-n_{\text{lag}}+1}, \mathbf{f}(\mathbf{s}_k)]^T \tag{11}$$

Three model variants are investigated:

1) **Linear NARX:** A ridge regression model with $\ell_2$ regularization applied directly to the augmented feature vector.
2) **Polynomial NARX:** Ridge regression applied to polynomial expansions of the input features, capturing nonlinear interactions up to a specified degree.
3) **Neural NARX:** A multilayer perceptron with rectified linear activation functions, trained via stochastic gradient descent with early stopping to prevent overfitting.

*1) Hyperparameter Optimization:* Given the number of potential hyperparameters, a systematic grid search with time series cross-validation is performed to identify optimal configurations. The search space includes:

- Number of lags: $n_{\text{lag}} \in \{2, 3, 5\}$
- Model type: $\{\text{linear}, \text{polynomial}, \text{neural}\}$
- Polynomial degree (if applicable): $\{2, 3\}$
- Neural network hidden layer configurations (if applicable)
- Regularization strength: $\alpha \in \{0.001, 0.01, 0.1\}$

Time series cross-validation is employed rather than random cross-validation to respect the temporal structure of the data [2]. The dataset is divided into sequential folds, where earlier data is used for training and subsequent data for validation, preventing information leakage from future to past observations.

*2) Selected Model:* Based on grid search results, the optimal configuration exhibits the following characteristics: a linear NARX model with two lags, ridge regularization strength of 0.01, utilizing input features from both past and present time steps. This configuration achieves root mean square error values of approximately $[0.128, 0.077, 0.808]$ meters per state component (horizontal position, vertical position, and orientation) on the validation dataset.

The selection of a linear model over nonlinear variants suggests that the relationship between laser features and state evolution is primarily linear within the operational regime studied. The modest input dimensionality ($2 \times 3 + 6 = 12$ features) and the strong performance without polynomial expansion or neural network complexity indicate that the essential predictive information is captured by the linear relationships encoded in the model coefficients.

The validation of the laser model reveals important insights into its predictive capabilities and limitations. Figure 4 presents a free-run simulation where the model operates autonomously, using only its own predictions as input for subsequent time steps. Starting from the initial reference pose $\mathbf{x}_0$, the model generates predictions iteratively according to:

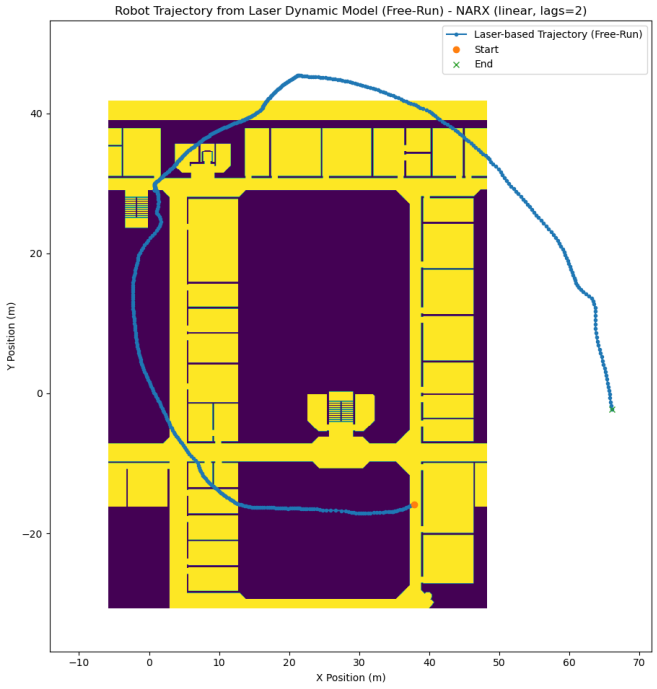


Fig. 4. Free-run simulation of the laser model.


Fig. 5. One-step ahead simulation of the laser model.

$$\hat{\mathbf{x}}_{k+1} = \mathbf{A}\hat{\mathbf{x}}_k + \mathbf{B}\mathbf{f}(\mathbf{s}_k) + \mathbf{b} \tag{12}$$

where $\hat{\mathbf{x}}_k$ denotes the model's own prediction at time step $k$. While this autonomous simulation succeeds in capturing the general trajectory morphology, it exhibits severe degradation in precision. The accumulated prediction errors compound over time, resulting in substantial deviations from the reference trajectory and rendering the free-run simulation unsuitable for direct pose estimation applications. The root mean square error grows progressively with prediction horizon, illustrating the accumulation of modeling uncertainty.

In contrast, Figure 5 demonstrates the performance of one-step ahead prediction, where the model receives the true state at each time step and predicts only the immediate next state:

$$\hat{\mathbf{x}}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{f}(\mathbf{s}_k) + \mathbf{b} \tag{13}$$

This one-step prediction exhibits markedly superior performance, with estimates closely tracking the reference trajectory throughout the experiment. The laser model achieves high fidelity when operating with access to ground truth state information, suggesting that the model has learned accurate local dynamics and that prediction errors primarily arise from accumulated drift rather than fundamental model inadequacy. This characteristic makes the laser model particularly well-suited as a measurement update component within filtering frameworks, where the filtering algorithms can provide corrected state estimates at each time step, thereby preventing error accumulation. The high one-step prediction accuracy validates the model's suitability for integration into the Unscented Kalman Filter, Extended Kalman Filter, and Particle Filter approaches discussed in Section VII.

## VI. Odometry Model

### A. Original Odometry Model

The odometry data, as detailed in Section III-C, is provided in the global reference frame with measurements expressed as cumulative displacements from the robot's initial pose. Given this characteristic, the original and most straightforward odometry model employs a static assumption wherein the robot's pose at any time step $k$ is computed by simply adding the odometry displacement to the initial pose.

Formally, let $\mathbf{x}_0 = [x_0, y_0, \theta_0]^T$ denote the initial pose of the robot and $\mathbf{o}_k = [\Delta x_k, \Delta y_k, \Delta\theta_k]^T$ represent the cumulative odometry displacement at time step $k$. The original odometry model is then defined as:

$$\mathbf{x}_k = \mathbf{x}_0 + \mathbf{o}_k \tag{14}$$

This linear, time-invariant model assumes that the odometry measurements accurately reflect the robot's true displacements without drift or systematic errors. In practice, however, odometry data accumulated over extended trajectories typically exhibits significant drift, as the errors in wheel encoders and other motion sensors compound over time. As discussed in Section I, this drift manifests as an increasing discrepancy

between the odometry-based estimates and the ground truth reference data, particularly over the several meters spanned by the experimental trajectory.

Despite its simplicity, this static model serves as a baseline for understanding the limitations of odometry-based localization and motivates the necessity of integrating laser measurements through advanced filtering techniques. The comparison between this original model's predictions and the reference data quantifies the magnitude of odometry drift and establishes the performance gap that the filtering algorithms aim to bridge through sensor fusion.

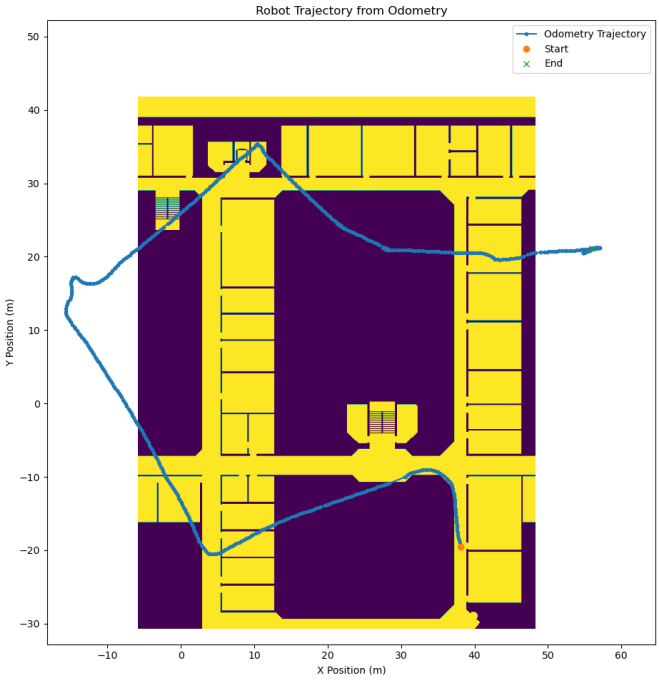


Fig. 6. Original odometry model free-run simulation.

Figure 6 demonstrates that the original static odometry model captures local movements of the robot with notable precision over short time intervals. However, the model exhibits pronounced drift when operated in free-run mode over the complete experimental trajectory. This drift accumulates progressively, resulting in substantial deviations from the reference trajectory as the mission duration extends. The root cause of this degradation lies in the inherent limitations of odometry-based localization: incremental errors from wheel encoders and inertial sensors compound over time, leading to systematic bias in the estimated pose.

Given that the odometry model serves as the *process model* in the filtering framework, as discussed in Section VI, the fidelity of odometry predictions directly influences the quality of the filtering estimates. A more accurate process model would reduce the prediction uncertainty and allow the filtering algorithms to operate more effectively within their respective correction mechanisms. This observation motivates the development of an improved odometry model that better captures the dynamics of robot motion and mitigates accumulated drift.

Despite the limitations of the original odometry model, an important insight emerges from the joint consideration of the odometry and laser models: even a modestly accurate laser model can effectively correct smaller pose mismatches introduced by odometry drift. The laser model, when integrated into a filtering framework, provides measurement updates that anchor the robot's estimated pose to environmental features encoded in the map. Since the laser measurements contain rich geometric information about the robot's surroundings, they can resolve ambiguities in the pose estimate that arise from odometry errors. Consequently, the combination of a corrected process model and laser-based measurement updates through advanced filtering techniques represents a promising approach for robust pose estimation, even when individual models exhibit limitations.

### B. Odometry Data Differentiation

To enhance the odometry model's fidelity and better capture the robot's motion dynamics, the cumulative odometry data is transformed into incremental displacements between consecutive time steps. This differentiation process allows the model to represent the robot's movement as a series of small, discrete steps rather than a single cumulative displacement from the initial pose. Formally, let $\mathbf{o}_k = [\Delta x_k, \Delta y_k, \Delta\theta_k]^T$ denote the cumulative odometry displacement at time step $k$. The incremental odometry displacement $\Delta\mathbf{o}_k$ between time steps $k-1$ and $k$ is computed as:

$$\Delta\mathbf{o}_k = \mathbf{o}_k - \mathbf{o}_{k-1} = [\Delta x_k - \Delta x_{k-1}, \Delta y_k - \Delta y_{k-1}, \Delta\theta_k - \Delta\theta_{k-1}]^T \tag{15}$$

This transformation yields a sequence of incremental displacements $\{\Delta\mathbf{o}_1, \Delta\mathbf{o}_2, \ldots, \Delta\mathbf{o}_N\}$ that more accurately reflects the robot's motion dynamics over time. By modeling the robot's pose evolution as a series of small steps, the odometry model can better account for variations in speed, direction changes, and other dynamic factors that influence the robot's trajectory.

## VII. Filters

### A. Unscented Kalman Filter

### B. Extended Kalman Filter

### C. Particle Filter

## VIII. Hyperparameter Optimization

## IX. Results

## X. Conclusion

## References


[1] L. A. Aguirre, *Introdução à identificação de sistemas: técnicas lineares e não lineares aplicadas a sistemas — teoria e aplicação*, 3ª ed. Belo Horizonte: Editora UFMG, 2007. ISBN 8570412207.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot and E. Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[3] T. Head, M. Kumar, H. Nahrstaedt, G. Louppe and I. Shcherbatyi, "scikit-optimize (skopt) v0.8.1," Python package, 2020. [Online]. Available: https://github.com/holgern/scikit-optimize.
[4] M. Carter, *FilterPy 1.4.4*. Python library for Kalman filtering and Bayesian estimation, 2022. [Online]. Available: https://github.com/mjcarter95/FilterPy.